Automated Reservation and Guidance System for Car Parking

Contents

Introduction and Market Research 4

Problem Statement 4

Business Level Requirement Analysis 5

Feature 1: Automatically detects, updates, and displays real-time parking slot availability using sensors, and prevents double booking through automated slot locking. 5

Feature 2: Allows users to reserve slots remotely via a mobile/web app, provides real-time guidance to the reserved or nearest slot, and supports contactless entry and exit 5

Feature 3: Facilitates secure digital payments, sends real-time notifications, and provides an admin dashboard for managing slot data, monitoring system performance, and generating reports 6

Component Level requirements of Feature 1 8

Design of Feature 1: 9

Class Diagram 9

Low level Test Cases and Traceability to Stakeholder's Requirements (SRs) 10

Component Level requirements of Feature 2 11

Design of Feature 2: 13

Class Diagram 13

Low level Test Cases and Traceability to Stakeholder's Requirements (SRs) 13

Component Level requirements of Feature 3 14

Design of Feature 3: 16

Class Diagram 16

Low level Test Cases and Traceability to Stakeholder's Requirements (SRs) 17

Project Plan 19

Tasks 19

Project deadlines 19

Task Assignment ..... **Error! Bookmark not defined.**

Software Design 20

Class Diagram 20

Flow Chart 20

Hardware Design 21

User Diagram ..... **Error! Bookmark not defined.**
Components Used 21
Test Cases 22
Code 30
References 30

Introduction and Market Research

In recent years, the rapid increase in the number of vehicles has led to significant urban challenges, one of the most pressing being the availability and management of parking spaces. Traditional parking systems are often inefficient, time-consuming, and contribute to traffic congestion and environmental pollution due to prolonged vehicle idling. To address these issues, the Automated Reservation and Guidance System for Car Parking offers a modern, intelligent solution that enhances parking efficiency through real-time monitoring, automated space allocation, and user-friendly booking features.

This system leverages technologies such as the Internet of Things (IoT), mobile applications, and real-time data processing to allow users to reserve parking spots in advance and receive guidance to the location of the available space. By integrating sensors and GPS-based navigation, the system reduces the time spent searching for parking, optimizes space utilization, and improves the overall user experience.

The global smart parking market is experiencing rapid growth, driven by increasing urbanization, rising vehicle ownership, and the need for efficient traffic management. According to industry reports, the market is projected to reach over USD 11 billion by 2028, growing at a CAGR of more than 10%. Urban drivers spend an average of 17–30 minutes searching for parking, leading to fuel waste, increased emissions, and economic losses. This has created strong demand for automated parking solutions that offer real-time space availability, advance reservation, and navigation assistance. The adoption of IoT, mobile apps, and sensor technologies further supports the development and deployment of such systems, making them essential for smart city infrastructure and commercial parking operations.

Problem Statement

Design and develop an automotive embedded system that have below features:

Feature 1: Automatically detects, updates, and displays real-time parking slot availability using sensors, and prevents double booking through automated slot locking.

Feature 2: Allows users to reserve slots remotely via a mobile/web app, provides real-time guidance to the reserved or nearest slot, and supports contactless entry and exit.

Feature 3: Facilitates secure digital payments, sends real-time notifications, and provides an admin dashboard for managing slot data, monitoring system performance, and generating reports.

Business Level Requirement Analysis

Feature 1: Automatically detects, updates, and displays real-time parking slot availability using sensors, and prevents double booking through automated slot locking.

Stakeholder Requirements (R1)	Business Test Cases / User Acceptance Test Case
R1.1: The system shall detect the presence or absence of a vehicle in each parking slot using sensors.	When a vehicle occupies a slot, the sensor detects it, and the system updates the slot status to "Occupied" in real time.
R1.2: The system shall update and display the real-time status of each slot (Available, Occupied, Reserved) on all user interfaces and display boards.	The status of a slot changes on the mobile app and display boards immediately when a vehicle enters or leaves a slot.
R1.3: The system shall lock a parking slot immediately when it is reserved to prevent double booking.	Upon reservation, the system locks the slot and prevents any other user from booking the same slot during the reservation period.
R1.4: The system shall lock a parking slot immediately when it is reserved to prevent double booking.	When the reservation time ends or the vehicle leaves, the slot unlocks and updates its status to "Available."
R1.5: The system shall ensure real-time synchronization of slot status across all user devices and displays.	Changes in slot status reflect simultaneously across all user devices and display units within 5 seconds.
R1.6: The system shall notify users attempting to book an already reserved or occupied slot.	If a user tries to book a reserved or occupied slot, the system displays an error message informing the slot is unavailable.

Feature 2: Allows users to reserve slots remotely via a mobile/web app, provides real-time guidance to the reserved or nearest slot, and supports contactless entry and exit

Stakeholder Requirements (R2)	Business Test Cases / User Acceptance Test Case
R2.1: The system shall allow users to view available parking slots and reserve a slot remotely via a mobile or web application.	User opens the app, views current available slots, and successfully reserves a slot remotely.
R2.2: The system shall provide real-time confirmation of the reservation, including reservation time and slot details.	After reservation, the user receives immediate confirmation with slot number, reservation time, and duration.
R2.3: The system shall provide navigation assistance to guide the user to the reserved or nearest available parking slot.	Upon reservation, the app provides step-by-step directions or a map guiding the user to the reserved or closest available slot.
R2.4: The system shall enable users to cancel or modify their reservation within a specified timeframe.	User cancels or modifies the reservation before the cutoff time; the system updates the slot status and notifies the user.
R2.5: The system shall send notifications and alerts related to reservation status, expiry, or slot changes to the user.	The user receives push notifications for booking confirmation, upcoming expiry, or slot status changes in real time.
R1.6: The system shall support multiple platforms (iOS, Android, Web) for booking and navigation.	The system allows successful booking and navigation on different platforms (mobile Android/iOS and web browsers).

Feature 3: Facilitates secure digital payments, sends real-time notifications, and provides an admin dashboard for managing slot data, monitoring system performance, and generating reports.

Stakeholder Requirements (R3)	Business Test Cases / User Acceptance Test Case
R3.1: The system shall support secure digital payment methods for parking fees, including credit/debit cards, mobile wallets, and other online payment gateways.	User completes a parking fee payment using different payment methods (credit card, mobile wallet) successfully and securely.
R3.2: The system shall automatically calculate parking fees based on reservation duration or usage.	System calculates the correct parking fee based on the user's reservation time and usage duration without errors.

R3.3: The system shall send real-time notifications to users for payment confirmations, booking alerts, expiry warnings, and system messages.	User receives timely notifications for payment confirmation, booking alerts, expiry warnings, and any system updates.
R1.4: The system shall provide an admin control panel to monitor parking slot status, user activity, payments, and system health.	Admin logs into the control panel and successfully views parking slot statuses, user activities, payment transactions, and system health metrics.
R1.5: The system shall allow administrators to manage slot availability, update pricing, and generate usage and financial reports.	Admin updates slot availability and pricing through the panel and generates accurate usage and financial reports.
R3.6: The system shall ensure data privacy and security in all transactions and user management operations.	All payment and user data transactions are encrypted and secure; penetration testing confirms data protection compliance.

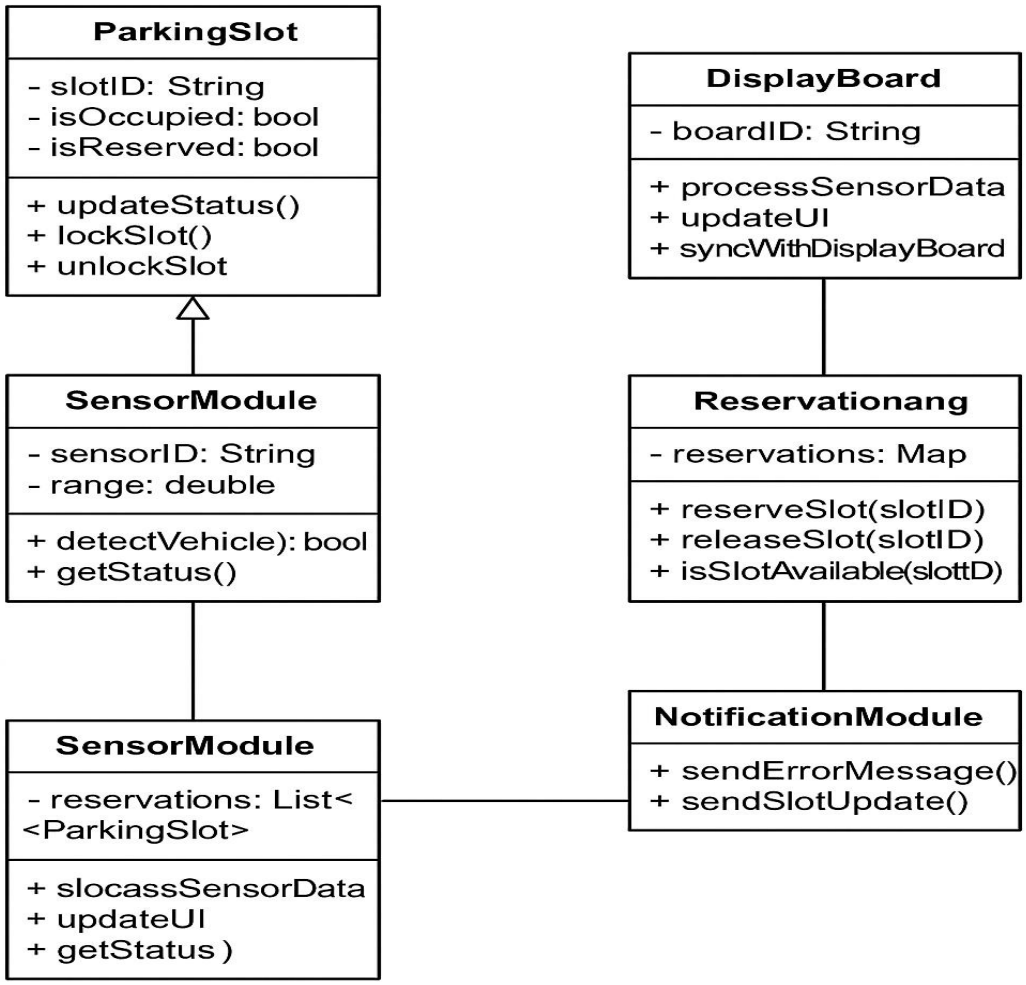
Component Level requirements of Feature 1

Stakeholder Requirements	Component	Requirements
R1.1: The system shall detect the presence or absence of a vehicle in each parking slot using sensors.	Parking Slot Sensor Module	Use ultrasonic or IR sensors with detection range of 2–3 meters and response time < 1 second; accuracy ≥ 95% under all conditions.
R1.2: The system shall update and display the real-time status of each slot (Available, Occupied, Reserved) on all user interfaces and display boards.	Central Processing Unit (CPU)	Process sensor input and update system database and UI within 2 seconds; status polling every 1 second.
	Display/Interface Module	Receive and show updated slot status within 3s latency.
R1.3: The system shall lock a parking slot immediately when it is reserved to prevent double booking**.**	Reservation Management Module	Lock slot in central database and mark as "Reserved" within 1 second of booking; deny parallel bookings with error code `409`.
	Control Unit (CPU)	Enforce lock command and manage conflicts between UI and DB.
R1.4: The system shall lock a parking slot immediately when it is reserved to prevent double booking.	Reservation Management Module	Automatically unlock and update slot status to "Available" within 2 seconds of expiry or sensor-detected vehicle exit.
	Control Unit (CPU)	Monitor timer/sensor and trigger slot release.

R1.5: The system shall ensure real-time synchronization of slot status across all user devices and displays.	Control Unit (CPU)	Collect slot changes and distribute updates across connected devices.
	Communication Interface Module	Broadcast status updates to connected devices (apps, display boards) every 3 seconds or on change event with ≤ 5s delay.
R1.6: The system shall notify users attempting to book an already reserved or occupied slot.	Control Unit (CPU)	Detect conflict and trigger appropriate user feedback.
	User Notification Module	Generate and deliver error notification to user UI within 2 seconds when a reserved/occupied slot is attempted for booking.

Design of Feature 1:

Class Diagram



Low level Test Cases and Traceability to Stakeholder's Requirements (SRs)

SR	Component	Method	Test Case Description	Input	Expected Output
R1.1	SensorModule	detectVehicle()	Verify sensor detects a vehicle accurately in an empty slot	No object in range	`true` (vehicle detected)
R1.1	SensorModule	detectVehicle()	Verify sensor does not detect when no vehicle is present	No object in range	`false` (no vehicle detected)

R1.2	ControlUnit	processSensorData ()	Verify control unit updates slot status when vehicle is detected	`detectVehicle()` returns `true`	`isOccupied` in `ParkingSlot` = `true`
R1.2	DisplayBoard	showSlotStatus()	Verify UI updates correctly with occupied status	Slot marked as occupied	Display shows slot as unavailable
R1.3	ReservationManager	releaseSlot(slotID)	Verify that reserved slot is locked immediately	Valid slot ID	Slot is locked; cannot be reserved again
R1.4	ReservationManager	releaseSlot(slotID)	Verify that a slot unlocks upon vehicle exit	Vehicle exits, timer expires	Slot becomes available
R1.5	ControlUnit	syncWithDisplayBoard()	Ensure status syncs in real-time with all connected boards	Change in slot status	Display boards update within 3 seconds
R1.6	NotificationModule	sendErrorMessage()	Alert user if trying to reserve already booked slot	User selects already reserved slot	"Slot not available" message shown

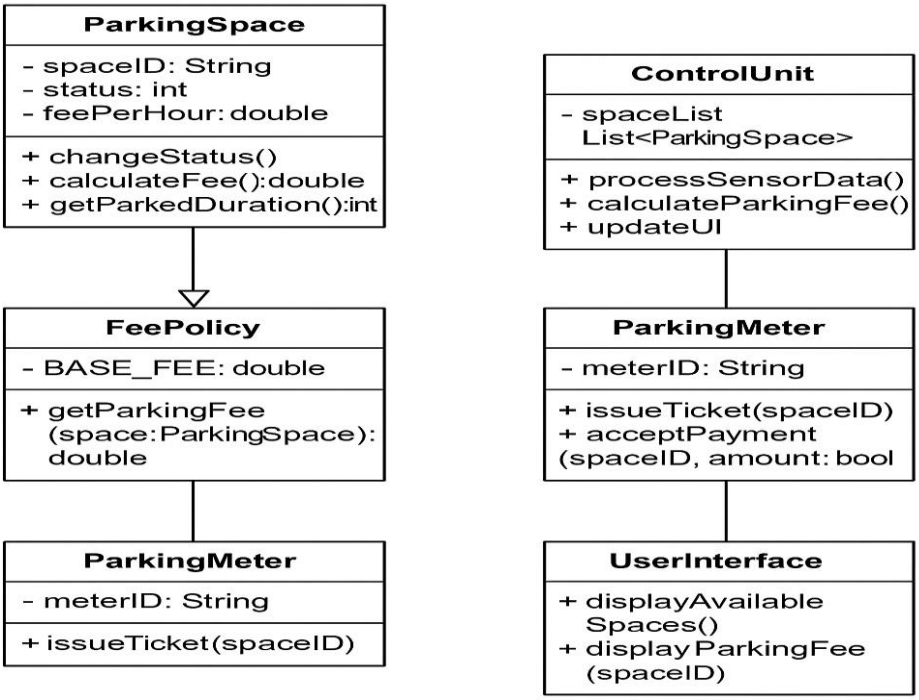
Component Level requirements of Feature 2

Stakeholder Requirements	Component	Requirements
R2.1: The system shall allow users to view available parking slots and reserve a slot remotely via a mobile or web application.	User Interface Module	Display real-time slot availability updated every 3 seconds; support reservation input with response time under 2 seconds.
	Control Unit (CPU)	Route user request to reservation module and respond with booking status.

R2.2: The system shall provide real-time confirmation of the reservation, including reservation time and slot details.	Reservation Management Module	Send reservation confirmation within 1 second of booking; include slot ID, start time, and reserved duration in the message.
	Control Unit (CPU)	Coordinate response and initiate lock command.
R2.3: The system shall provide navigation assistance to guide the user to the reserved or nearest available parking slot.	Navigation Module	Provide navigation instructions with accuracy within 1 meter indoors; update route dynamically every 5 seconds.
	Control Unit (CPU)	Fetch slot location and relay to navigation interface.
R2.4: The system shall enable users to cancel or modify their reservation within a specified timeframe**.**	Reservation Management Module	Allow cancellation/modification up to 15 minutes before reservation start time; update slot status within 2 seconds of change.
	Control Unit (CPU)	Handle modification logic and re-sync status across all systems.
R2.5: The system shall send notifications and alerts related to reservation status, expiry, or slot changes to the user.	Notification Module	Deliver push notifications within 3 seconds of event trigger (e.g., reservation confirmation or expiry warning).
	Control Unit (CPU)	Detect event triggers and initiate alert dispatch.
R1.6: The system shall support multiple platforms (iOS, Android, Web) for booking and navigation.	Cross-Platform Support Module	Ensure UI responsiveness < 200 ms on devices running iOS 13+, Android 9+, and latest web browsers (Chrome, Firefox, Safari).
	Control Unit (CPU)	Interface with APIs and adapt output formats based on platform.

Design of Feature 2:

Class Diagram



Low level Test Cases and Traceability to Stakeholder's Requirements (SRs)

SR	Component	Method	Test Case Description	Input	Expected Output
R1.1	Slot Reservation	viewAvailableSlots()	Verify available slots are displayed on mobile/web app	User logs into mobile/web app and requests available slots	List of available slots with status (available/reserved) displayed correctly
R1.1	Slot Reservation	reserveSlot(slotID)	Reserve a specific available parking slot remotely	User selects an available slotID and submits reservation request	Reservation success message with slot details

R1.2	Reservation System	confirmReservation()	Confirm reservation in real-time.	Reservation request is successfully submitted	Real-time confirmation with reservation time and slot details shown
R1.3	Navigation	provideNavigation(slotID)	Provide navigation to reserved slot	User selects reserved slot or nearest available slot for navigation	Navigation instructions (route, ETA) displayed correctly
R1.4	Reservation System	cancelReservation(resID)	Cancel a reservation within allowed timeframe	User requests to cancel reservation before deadline	Cancellation success message, slot marked available
R1.4	Reservation System	modifyReservation(resID)	Modify reservation within allowed timeframe	User modifies reservation details (time/slot) within allowed period	Modification success confirmation and updated reservation details
R1.5	Notification System	sendNotification()	Send notifications on reservation status changes	Reservation status changes (confirmed, cancelled, expired)	User receives relevant notification/alert on device
R1.6	Cross-Platform	accessViaPlatform(platform)	Access booking and navigation on different platforms	User logs in via iOS, Android, or Web app	Booking and navigation features function correctly on all supported platforms

Component Level requirements of Feature 3

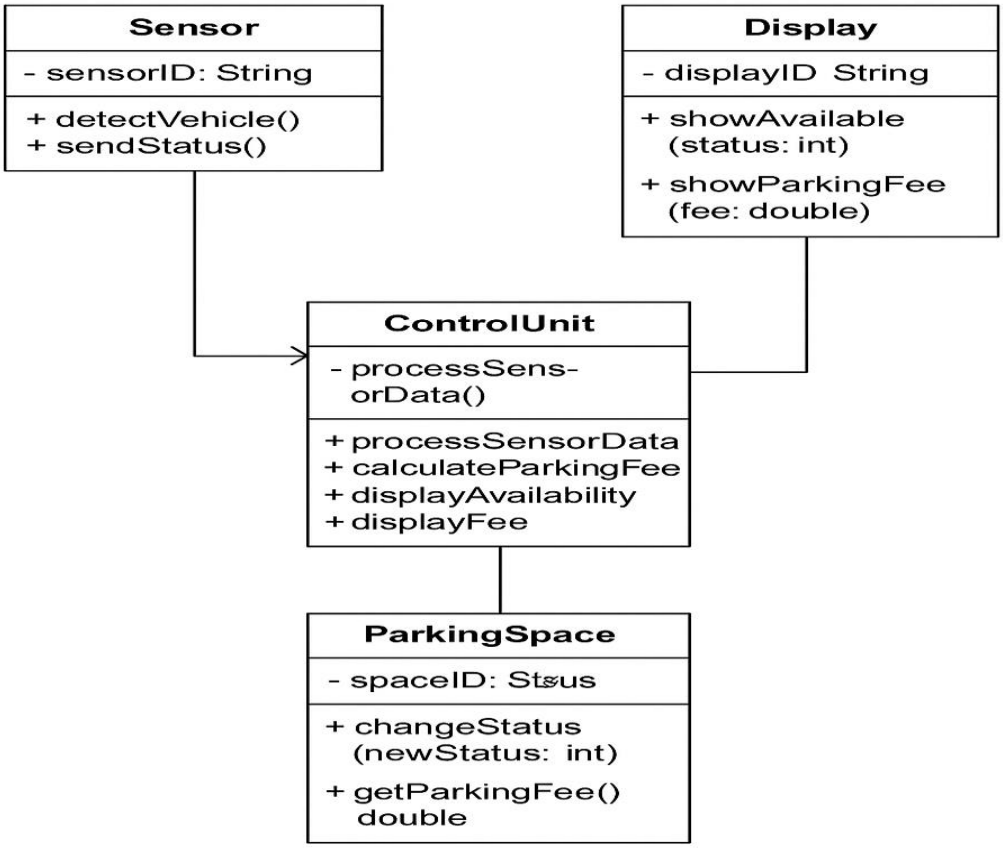
Stakeholder Requirements	Component	Requirements
R1.1: The system shall support secure digital payment methods for parking fees, including credit/debit cards, mobile wallets, and other online payment gateways**.**	Payment Gateway Module	Integrate APIs for PayPal, UPI, credit/debit cards; ensure PCI-DSS compliance; transaction timeout < 10 seconds.

	Control Unit (CPU)	Validate payment details, communicate with gateway, log results securely.
R1.2: The system shall automatically calculate parking fees based on reservation duration or usage.	Billing Engine	Calculate fees based on ₹20/hour rate; round to nearest 15 minutes; auto-generate invoice within 1 second post-checkout.
	Control Unit (CPU)	Compute duration from timestamps; apply fee logic.
R1.3: The system shall send real-time notifications to users for payment confirmations, booking alerts, expiry warnings, and system messages.	Notification Module	Send confirmation SMS/email/push notification within 3 seconds of payment or reservation event.
	Control Unit (CPU)	Trigger notifications based on payment status or event changes.
R1.4: The system shall provide an admin control panel to monitor parking slot status, user activity, payments, and system health.	Admin Dashboard Module	Dashboard shall refresh data every 10 seconds; show real-time occupancy, user logs, payment history, and system health metrics.
	Control Unit (CPU)	Aggregate system data and feed real-time stats to dashboard.
R1.5: The system shall allow administrators to manage slot availability, update pricing, and generate usage and financial reports.	Admin Management Module	Admin can update pricing slabs, activate/deactivate slots, and export usage and revenue reports in CSV or PDF within 5 seconds.
	Control Unit (CPU)	Authorize changes and push updates system-wide.

R1.6: The system shall ensure data privacy and security in all transactions and user management operations.	Security and Encryption Module	Use AES-256 for data encryption; enable HTTPS across all endpoints; enforce OAuth 2.0 for user/admin authentication.
	Control Unit (CPU)	Enforce authentication, handle encrypted data routing and logging.

Design of Feature 3:

Class Diagram



Low level Test Cases and Traceability to Stakeholder's Requirements (SRs)

SR	Component	Method	Test Case Description	Input	Expected Output
R1.1	Payment Gateway	processPayment()	Verify user can pay using credit/debit card	Payment method: Credit Card, Amount: [validAmount]	Payment success message with transaction ID
R1.1	Payment Gateway	processPayment()	Verify user can pay using mobile wallet	Payment method: Mobile Wallet, Amount: [validAmount]	Payment success message and receipt
R1.2	Fee Calculator	calculateFee(startTime, endTime)	Calculate fee based on reservation time	Start time: [startTime], End time: [endTime]	Total fee = [calculatedAmount]
R1.2	Fee Calculator	calculateFee(duration)	Calculate fee for extended parking time	Duration: [totalDuration], Rate: [ratePerHour]	Total fee = [baseFee + overstayFee]
R1.3	Notification System	sendNotification(eventType)	Send notification for payment confirmation	Event: PaymentSuccess, Reservation ID: [resID]	Message: "Payment confirmed for reservation [resID]"
R1.3	Notification System	sendNotification(eventType)	Send alert before reservation expiry	Event: ExpiryAlert, Time remaining: [timeRemaining]	Notification: "Your reservation will expire in [timeRemaining]"
R1.4	Admin Dashboard	viewDashboard()	Display real-time system status to admin	User role: Admin	Dashboard displays parking occupancy, revenue, active sessions
R1.4	Admin Dashboard	monitorHealth()	Display current system performance metrics	Trigger: Admin selects "System Health" tab	Display includes CPU usage, DB status, uptime
R1.5	Admin Management	updatePricing(rate)	Update pricing details	New hourly rate: [ratePerHour]	Confirmation message: "Pricing updated successfully"
R1.5	Admin Management	generateReport(type, period)	Generate report for given time period	Report type: [financial/usage], Period:	Generated report in PDF/CSV format

				[startDate – endDate]	
R1.6	Security Layer	encryptTransaction(data)	Validate encryption of transaction data	Transaction data: [paymentDetails]	Encrypted payload, transmitted over secure protocol (e.g., HTTPS)
R1.6	Security Layer	encryptTransaction(data)	View admin action logs	User role: Admin, Filter: [dateRange]	Log entries with timestamps, user actions, and affected components

Project Plan

Tasks

Whole project can be divided into 3 Tasks based on the features that are implemented

1. Real-time Slot Detection and Slot Locking
2. Remote Reservation and Navigation
3. Admin Dashboard

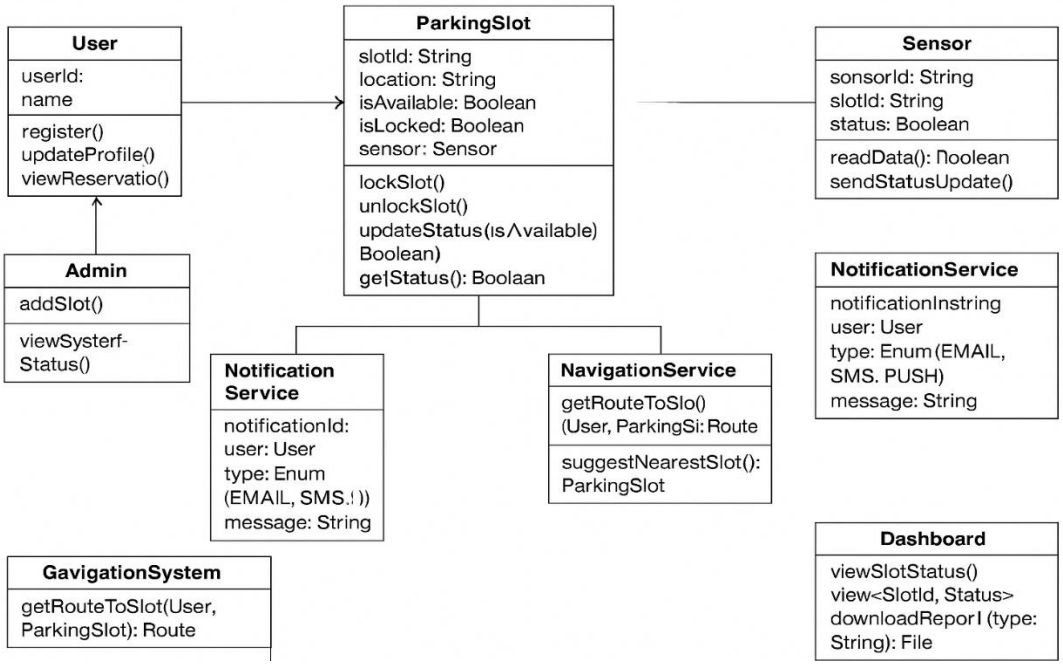
Project deadlines

Sl. No	Task	Status
1	Market Research and adjusting the problem statement	**Completed**
2	Finalizing the Features	**Completed**
3	Finalizing the individual Requirements and Business Test Cases	**Completed**
4	Individual Implementation and Component Level Testing	**Completed**
5	Integration and Testing	**Completed**
6	Documentation	**Completed**
7	Presentation	**Completed**
8	Complete Documentation	**Completed**

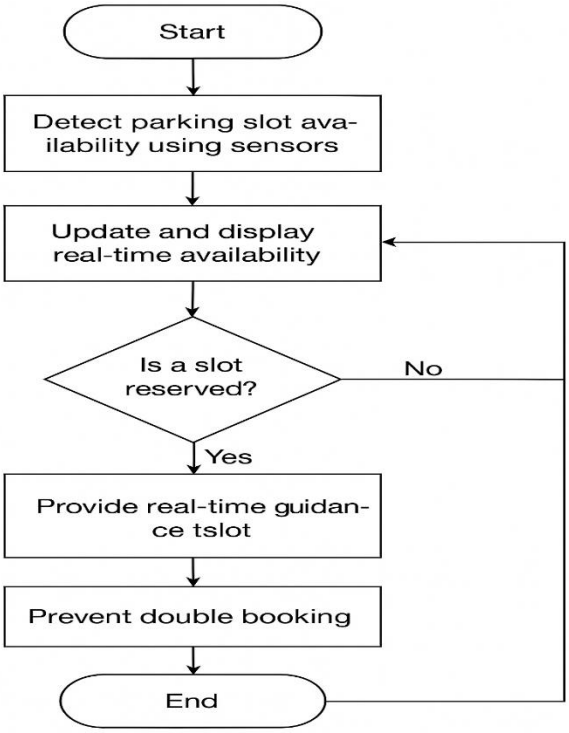
Table 1: Project Deadlines

Software Design

Class Diagram



Flow Chart



Hardware Design

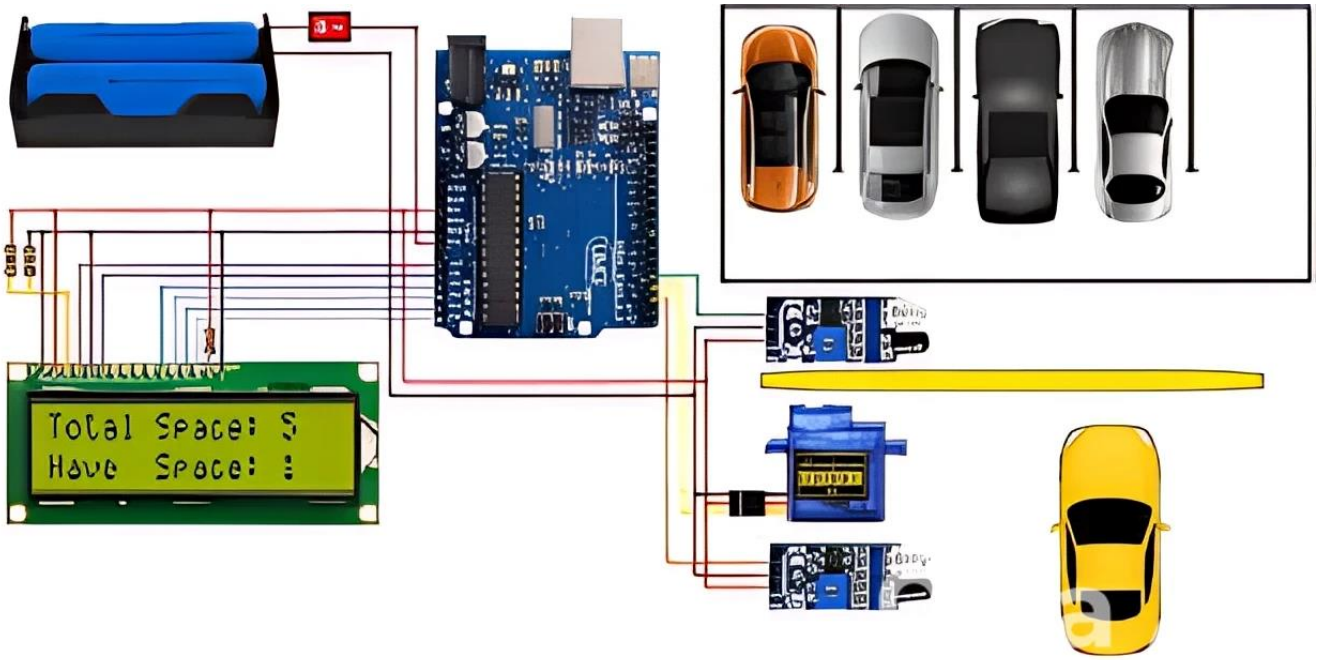


Figure 5: Final Ciruit (Integrated)

Components Used

Component	Quantity	Name
Arduino UNO	1	U1
20*4 LCD Display	1	U2
I2C Module for the LCD	1	U3
IR Sensors	7	U4, U5, U6, U7, U8, U9, U10
Mini Servo Motor SG-90	1	SERVO1
Power Source	1	U10

Table 2: Bill of Materials

Test Cases

Test ID	Description/Objective	INPUT	Expected O/P	Actual O/P	Result
1	Test successful reservation via terminal	User enters vehicle number, selects slo	Reservation successful, slot marked as reserved	Reservation successful	Pass

		t, confirms			
2	Test reservation when all slots are full	Attempt reservation when all slots are occu	Reservation denied, message: “No slots available”	“No slots available"	Pass

		pied			
3	Test sensor detection for empty slot	Car leaves slot, sensor sends signal	Slot status updates to "Available"	Slot marked available	Pass
4	Test sensor detection for occupied slot	Car enters slo	Slot status updates to "Occupied"	Slot marked oc	Pass

		t, sensor triggers		cupied	
5	Test guidance display after reservation	User reserves Slot A2	Display shows "Go to Slot A2"	"Go to Slot A2"	Pass
6	Test barrier gate opens with valid reservation	Vehicle arrive	Barrier opens	Barrier opened	Pass

		swith valid reservation			
7	Test barrier gate denied for no reservation	Vehicle arrives swith out reserv	Barrier remains closed, error message displayed	Access denied	Pass

		a t i o n			
8	Test system behavior with sensor failure	S e n s o r d o e s n o t s e n d s i g n a l (h a r d w a r e f a u l t)	System logs error, marks slot as “Unavailable”	S l o t fl a g g e d f a u lt y	P a s s
9	Test parking slot occupancy when sensor is blocked manually	S e n	System flags as suspicious or requests manual check	F l a	P a

		sorisblocked(fakeoccupancy)		ggedcorrectly	ss
10	Test system reboot with active reservations	Restartserverwhilesl	Data persists, system resumes with same status	Staterecovered	Pass

		otsarereserved			

Code

```
# include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

/*
  This project is an Automated Reservation and Guidance System for Car Parking.
  It uses IR sensors to detect vehicle presence in parking slots and at entrance/exit points.
  A servo motor is used as the gate barrier.
  The system displays the availability of slots on an I2C LCD.
*/

LiquidCrystal_I2C lcd(0x27, 20, 4);  /* Initialize 20x4 I2C LCD at address 0x27 */
Servo myservo;                       /* Servo object to control the gate */

/* Define pin numbers for IR sensors */
#define ir_enter 2     /* IR sensor at entry gate */
#define ir_back  4     /* IR sensor at exit gate */
#define ir_car1 5      /* Sensor for parking slot 1 */
#define ir_car2 6      /* Sensor for parking slot 2 */
#define ir_car3 7      /* Sensor for parking slot 3 */
#define ir_car4 8      /* Sensor for parking slot 4 */
#define ir_car5 9      /* Sensor for parking slot 5 */
#define ir_car6 10     /* Sensor for parking slot 6 */

/*
  Variables to track slot occupancy:
  Value 1 = Occupied
  Value 0 = Available
*/
```

```
int S1=0, S2=0, S3=0, S4=0, S5=0, S6=0;

/* Flags to indicate entry/exit events */
int flag1=0;   /* Entry flag */
int flag2=0;   /* Exit flag */

int slot = 6;  /* Total available slots */

void setup() {
  Serial.begin(9600);  /* Initialize serial communication for debugging */

  /* Set all sensor pins as input */
  pinMode(ir_car1, INPUT);
  pinMode(ir_car2, INPUT);
  pinMode(ir_car3, INPUT);
  pinMode(ir_car4, INPUT);
  pinMode(ir_car5, INPUT);
  pinMode(ir_car6, INPUT);
  pinMode(ir_enter, INPUT);
  pinMode(ir_back, INPUT);

  myservo.attach(3);      /* Connect servo to pin 3 */
  myservo.write(90);      /* Initial gate position: closed (90 degrees) */

  lcd.begin(20, 4);       /* Begin LCD interface with 20x4 character size */
  lcd.setCursor(0, 1);
  lcd.print("    Car  parking  ");

lcd.setCursor(0, 2);
  lcd.print("       System     ");

  delay(2000);            /* Show welcome message for 2 seconds */
  lcd.clear();            /* Clear the LCD */
```

```
  /* Read sensor values and update available slots */
  Read_Sensor();
  int total = S1 + S2 + S3 + S4 + S5 + S6;  /* Count occupied slots */
  slot = slot - total;                      /* Calculate free slots */
}

void loop() {
  Read_Sensor();  /* Refresh sensor readings */

  /* Display total available slots */
  lcd.setCursor(0, 0);
  lcd.print("   Have Slot: ");
  lcd.print(slot);
  lcd.print("    ");

  /* Display status of each slot */
  lcd.setCursor(0, 1);
  lcd.print(S1 == 1 ? "S1:Fill " : "S1:Empty");

  lcd.setCursor(10, 1);
  lcd.print(S2 == 1 ? "S2:Fill " : "S2:Empty");

  lcd.setCursor(0, 2);
  lcd.print(S3 == 1 ? "S3:Fill " : "S3:Empty");

  lcd.setCursor(10, 2);
  lcd.print(S4 == 1 ? "S4:Fill " : "S4:Empty");

  lcd.setCursor(0, 3);
  lcd.print(S5 == 1 ? "S5:Fill " : "S5:Empty");

  lcd.setCursor(10, 3);
  lcd.print(S6 == 1 ? "S6:Fill " : "S6:Empty");
```

```
  /*
    Entry logic:
    If the IR sensor at the entrance is triggered and flag1 is not already set,
    and slots are available, open the gate and reduce available slot count.
  */

 if (digitalRead(ir_enter) == 0 && flag1 == 0) {
    if (slot > 0) {
      flag1 = 1;
      if (flag2 == 0) {
        myservo.write(180);   /* Open gate */
        slot = slot - 1;      /* One car entered */
      }
    } else {
      /* No slot available */
      lcd.setCursor(0, 0);
      lcd.print(" Sorry Parking Full ");
      delay(1500);
    }
  }

  /*
    Exit logic:
    If IR sensor at the exit is triggered and flag2 is not already set,
    and there was no car entry, then increase the available slots.
  */

 if (digitalRead(ir_back) == 0 && flag2 == 0) {
    flag2 = 1;
    if (flag1 == 0) {
      myservo.write(180);  /* Open gate */
      slot = slot + 1;     /* One car exited */
    }
  }
```

```
  /*
    Close gate:
    If both entry and exit are flagged (meaning transition is complete), close gate.
  */

if (flag1 == 1 && flag2 == 1) {
    delay(1000);          /* Wait before closing gate */
    myservo.write(90);     /* Close gate */
    flag1 = 0;
    flag2 = 0;
  }

  delay(1);  /* Small delay for smooth processing */
}

/*
  Function: Read_Sensor
  Purpose: Read all 6 parking slot IR sensors and update S1 to S6.
  These values indicate whether each slot is filled or empty.
*/
void Read_Sensor() {
  S1 = (digitalRead(ir_car1) == 0) ? 1 : 0;
  S2 = (digitalRead(ir_car2) == 0) ? 1 : 0;
  S3 = (digitalRead(ir_car3) == 0) ? 1 : 0;
  S4 = (digitalRead(ir_car4) == 0) ? 1 : 0;
  S5 = (digitalRead(ir_car5) == 0) ? 1 : 0;
  S6 = (digitalRead(ir_car6) == 0) ? 1 : 0;
}
```

References

1. Kumar, S., & Pandey, S. (2018). ***Smart Car Parking System Using IoT*. *International Journal of Engineering and Technology*, 7(2.3), 125-128. https://doi.org/10.14419/ijet.v7i2.3.12259**
2. IEEE Standard 802.15.4 (2020). ***Low-Rate Wireless Personal Area Networks (LR-WPANs) for Sensor Networks*** **https://standards.ieee.org/standard/802_15_4-2020.html**
3. Arduino Documentation**. *Arduino UNO Hardware and Programming Reference*** **https://www.arduino.cc/en/Guide**
4. HC-SR04 Ultrasonic Sensor Datasheet**. Used for real-time parking slot detection. https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf**
5. Raspberry Pi Documentation**. For reservation server and processing unit https://www.raspberrypi.com/documentation/**
6. Espressif Systems (2022). ***ESP8266 WiFi Module Technical Reference*. https://www.espressif.com/en/products/socs/esp8266**
7. Google Firebase**. For cloud database and real-time slot syncing https://firebase.google.com/docs**
8. Smart Parking: A Literature Review (2017) **A. Khanna & R. Anand *International Journal of Scientific and Research Publications*, Volume 7, Issue 4 http://www.ijsrp.org/research-paper-0417/ijsrp-p64112.pdf**